



Opisi algoritama

Zadatke, testne primjere i rješenja pripremili: Fran Babić, Toni Brajko, Nikola Dmitrović, Sofija Velkovska i Jakov Celin.

Primjeri implementiranih rješenja dani su u priloženim izvornim kodovima.

Zadatak Sljedbenik

Pripremio: Nikola Dmitrović

Potrebno znanje: naredbe učitavanja i ispisivanja

Nakon što je čuo broj n , Perica je izgovorio sljedbenika broja n , to jest prirodni broj koji je za jedan veći od n . Dakle, dovoljno je učitati broj n s ulaza, a zatim ispisati $n + 1$.

Programski kod (pisan u Python 3):

```
N = int(input())
```

```
print(N+1)
```

Zadatak Zakon

Pripremio: Nikola Dmitrović

Potrebno znanje: naredba odlučivanja (if)

Ivica je poštovao zakon ako *nije* vozio kroz naseljeno mjesto strogo brže od 50 kilometara na sat. To znači da je vozio brzinom manjom ili jednakom 50. Učitamo broj x , koji predstavlja brzinu kojom je Ivica vozio kroz naseljeno mjesto. Ako je $x \leq 50$, Ivica je poštovao zakon, pa ispisujemo DA. Inače, Ivica nije poštovao zakon i ispisujemo NE.

Programski kod (pisan u Python 3):

```
X = int(input())
```

```
if X <= 50:  
    print("DA")  
else:  
    print("NE")
```



Zadatak Smijeh

Pripremio: Jakov Celin

Potrebno znanje: naredba odlučivanja (if), naredba ponavljanja (for)

Odredimo prvo za neki broj, hoće li se Vili nasmijati kada ga pročita. Za dobivanje zadnje znamenke broja možemo uzeti broj ostatak pri djeljenu s 10. Tada s naredbom odlučivanja (if) provjerimo je li zadnja znamenka jednaka 6 ili 7. Ovaj postupak ponovimo za svih N upisanih brojeva koristeći naredbu ponavljanja (for petlju). Na kraju ispišemo brojač kojim smo brojali koliko puta se Vili nasmijao.

Programski kod (pisan u Python 3):

```
N = int(input())
BR = 0

for i in range(N):
    X = int(input())
    if (X % 10 == 6 or X % 10 == 7):
        BR += 1

print(BR)
```

Zadatak Rima

Pripremila: Sofija Velkovska

Potrebno znanje: stringovi, naredba ponavljanja (for), naredba odlučivanja (if)

Prije svega, moramo pronaći način kako utvrditi rimuju li se dva stiha. Promatrat ćemo posljednje riječi svakog stiha; nazvat ćemo ih a i b . Najprije moramo provjeriti jesu li oba stringa duljine barem k . Ako su a ili b sastavljeni od manje od k znakova, ova dva stiha se ne rimuju i ne bismo trebali nastavljati s daljnjim provjerama. Ako su oba niza dovoljno duga, trebamo provjeriti podudaraju li se njihova posljednja k slova. Označimo duljine stringa a i b s a_len i b_len , tada trebamo usporediti slovo na poziciji $a_len - k$ u a sa slovom na poziciji $b_len - k$ u b , slovo na poziciji $a_len - k + 1$ u a sa slovom na poziciji $b_len - k + 1$ u b itd. sve do slova na pozicijama $a_len - 1$ u a i $b_len - 1$ u b (zadnja slova u stringovima). Ako se u bilo kojem od ovih parova slova razlikuju, stihovi se ne rimuju.

Postoje i drugi načini za provjeru rimuju li se dva stiha. Na primjer, možemo koristiti $a[-k:]$ i $b[-k:]$ u Pythonu, ili $a.substr(a_len - k, k)$ i $b.substr(b_len - k, k)$ u C++, kako bismo dobili posljednjih k slova riječi, a zatim ih možemo usporediti s operatorom `"=="`. Imajte na umu da još uvijek moramo provjeriti je li duljina i stringa a i stringa b barem k prije izvođenja ovih operacija.

Sada kada imamo algoritam koji će nam reći rimuju li se dva stiha ili ne, možemo ga koristiti za provjeru vrsta rime koje se pojavljuju u svakoj strofi. Želimo obrađivati pjesmu strofu po strofu, a za to možemo koristiti petlju *for* koja ide od prvog stiha do posljednjeg s korakom +4 (`for (int i = 0; i < n; i = i + 4)`). Recimo da smo trenutno na stihu i , strofa u kojoj se nalazi i sastoji se od stihova i , $i + 1$, $i + 2$ i $i + 3$.

Da bismo provjerili javlja li se parna rima u određenoj strofi, provjeravamo rimuju li se stihovi i i $i + 1$, te stihovi $i + 2$ i $i + 3$ koristeći postupak koji smo gore objasnili. Ako se oba para stihova rimuju, strofa ima parnu rimu. Ponavljamo postupak za ostale vrste rime, pri čemu stihove uparujemo na drugačiji način. Za unakrsnu rimu sparimo i s $i + 2$ i $i + 1$ s $i + 3$, a za obgrljenu sparimo i s $i + 3$ i $i + 1$ s $i + 2$. Moramo pratiti broj strofa u kojima se pojavljuje svaka vrsta rime, a to možemo učiniti tako da ćemo imati tri različite cjelobrojne varijable, jednu za svaku vrstu rime, koje povećavamo svaki put kada strofa zadovoljava kriterije za tu vrstu rime.



Zadatak Krugomet

Pripremio: Jakov Celin

Potrebno znanje: binary lifting

Primjetimo da osoba i uvijek baca sve svoje loptice osobi s_i , zatim osoba s_i te loptice (i ostale dobivene, ali njih sada ignoriramo) baca osobi s_{s_i} , zatim s_{s_i} baca... Tvrdnja nas navodi da za svaku osobu i odredimo gdje će nakon K koraka završiti loptice koje su se na početku igre nalazile baš kod osobe i . Zatim bi toj osobi pridodali loptice osobe i te na kraju pogledali tko ima najviše loptica i ispisali njihov popis.

Ostaje samo pronaći osobu kod koje će završiti loptice od osobe i .

Za $K = 1$ to je osoba s_i .

za $K = 2$ to je osoba s_{s_i} .

Za $K = 3$ to je osoba $s_{s_{s_i}}$.

Nazovimo X -tom simpatijom osobe i osobu do koje stignemo tako da X puta zaredom "skočimo" na osobu koju određuje niz s , počevši od osobe i i svaki put skačući s trenutno dosegnute osobe. Slijedi da će za broj koraka K , sve loptice od osobe i završiti kod njene K -te simpatije.

Za 14 bodova bilo je dovoljno s for petljom samo simulirati ovaj postupak. Za dodatnih 26 bodova, bilo je potrebno permutaciju rastaviti na cikluse te pomoću broja K i duljina ciklusa odrediti sve K -te simpatije. Zadatak se za sve bodove mogao riješiti na mnoge načine koji su koristili strukturu grafa simpatija, no ona su implementacijski i idejno zahtjevnija od službenog pa ćemo ovdje opisati samo službeno rješenje koje koristi samo algoritam zvan binary lifting.

Ako za potenciju broja 2 znamo 2^x -tu simpatiju svake osobe, tada možemo pronaći K -tu simpatiju bilo koje osobe u složenosti $O(\log K)$. Zapišemo li K u binarnom zapisu (sastoji se od točno $\log K + 1$ znamenki) tada napravimo skokove samo na 2^x -te simpatije gdje su x pozicije jedinica u binarnom zapisu broja K .

Preostaje još samo izračunati 2^x -te simpatije svake osobe za $1 \leq x \leq 30$ jer je $1 \leq K \leq 10^9$. Primjetimo da je $2^x = 2^{x-1} + 2^{x-1}$ tj. skok na 2^x -tu simpatiju možemo prikazati kao skok na 2^{x-1} -u simpatiju pa s nje napraviti skok na njenu 2^{x-1} -u simpatiju. Ako znamo sve prve simpatije, pomoću prethodne tvrdnje možemo izračunati sve druge simpatije, a pomoću njih sve četvrte... Vremenska i memorijska složenost opisanog algoritma je $O(n \log K)$. Za detalje, pogledajte službenu implementaciju.

Zadatak Harmonija

Pripremio: Jakov Celin

Potrebno znanje: binary lifting, LCA, matrice, dinamičko programiranje

Primjetimo da iz uvjeta harmoničnog puta, u svakom trenutku puta između neka 2 čvora, razlika između crvenih i plavih boja mora ≤ 2 tj. $-2 \leq \text{brojCrvenihČvorova} - \text{brojPlavihČvorova} \leq 2$. To nas navodi na rješenje da put od čvora A do čvora B pretvorimo u niz te njega riješimo dinamičkim programiranjem.

Neka $dp[i][\text{raz}]$ predstavlja maksimalnu vrijednost harmoničnog puta od A do i -tog čvora po redu na putu od A do B, a da je trenutna razlika boja jednaka raz . Granamo se na 2 prijelaza (biramo hoće li i -ti čvor biti crven ili plav) te provjerimo hoće li se razlika boja nalaziti između -2 i 2 nakon odabira. Ovaj algoritam donosi 68 bodova zbog prevelike vremenske složenosti $O(nq)$.

Sve bodove nosila su mnoga rješenja koja su koristila razne tehnike (centroidna dekompozicija, HLD...), no opisati ćemo najjednostavnije.

Označimo najveću vrijednost harmoničnog puta od čvora A do čvora B koji je započeo s razlikom boja x , a završio s razlikom boja y kao AB_{xy} . Primjetimo da ako se čvor B nalazi na putu između čvorova A i D te ako je čvor C prvi čvor na putu od A do D nakon B, onda vrijedi $AB_{xy} + CD_{yz} = AD_{xz}$. Ova tvrdnja nam omogućava spajati "najbolje" harmonične puteve s istim krajem u duže.



Glavna ideja je za lance duljina potencija broja 2 pamti 5x5 matricu koja predstavlja s kojom razlikom smo ušli u lanac i s kojom ćemo izaći. Upit na putu od A do B rješavamo tako da put podijelimo na $O(\log n)$ disjunktne lanaca pomoću običnog binary lifting/LCA algoritma te da lance spajamo na sličan način kao što bi množili matrice koristeći prethodne tvrdnje. Pažljivom implementacijom (npr. lance od LCA do B moramo obrnuto spajati pri upitu i pri njihovom računanju) ovog algoritma možemo ostvariti sve bodove na ovom zadatku, a za detalje pogledajte službenu implementaciju.

Zadatak Kraljica

Pripremio: Jakov Celin

Potrebno znanje: 0-1 BFS

Za jednostavnije računanje složenosti koristiti ćemo tvrdnju da je u najgorem slučaju $N = M = 1000$ pa ćemo pisati da je $N = M$, iako to ne mora vrijediti.

Prvi i treći podzadatak mogli su se riješiti korištenjem običnog BFS algoritma koji se s trenutnog polja kraljice pokušavao širiti na sva moguća polja na koja kraljica može skočiti. Vremenenska složenost takvog pristupa bila je $O(N^3)$.

Drugi i četvrti podzadatak zahtijevaju znanje 0-1 BFSa, a četvrti implementaciju portala uz obične prijelaze kraljice. Glavna observacija koja nas dovodi do rješenja jest da za svako polje pamtimo i iz kojeg smjera smo prethodno došli u to polje tj. stanje nam neće biti samo redak i stupac polja matrice nego i smjer iz kojega smo u to polje došli. Sada se iz stanja (smjer, polje), gdje je smjer broj od 0 do 8, a polje su indeksi retka i stupca granamo u 8 neposrednih susjeda (ako je to moguće) i portal (ako on postoji na našem polju). Težine prijelaza su 0 ili 1 ovisno o tome jesmo li zadržali isti smjer (ne radimo dodatan potez, težina 0) kao i prije ili smo ga promijenili (moramo napraviti novi potez, težina 1).

Algoritam koji računa najkraći put između 2 čvora grafa (stanja) gdje su težine bridova jednake 0 ili 1 zovemo 0-1 BFS te se njegova implementacija nalazi u priloženom kodu. Zbog jednostavnije implementacije, stanje prelaska u portal označeno je smjerom 9 jer je uz to implementacija potpuno ista implementaciji drugog podzadatka bez portala. Složenost algoritma je $O(NM)$, no uz veliku konstantu (svako od $9NM$ stanja se grana u 8 susjeda, a ponekad i u portal). Za detalje, pogledajte službenu implementaciju.

Zadatak Zagi

Pripremio: Fran Babić

Potrebno znanje: Teorija igara, Sprague-Grundy teorem, ad-hoc, memoizacija rekurzije

U slučaju kada je $a_i \leq 2$, nakon što prvi igrač odigra potez tj. izabere 1 ili 2, preostali nizovi će se sastojati od međusobno jednakih brojeva. Zaključujemo da će se igra završiti u točno onoliko poteza koliko je podnizova preostalo, a tko pobjeđuje ovisi o parnosti. Prvi igrač može birati tj. izabrati će broj takav da je broj preostalih nizova paran. Ako takav postoji, pobijediti će prvi igrač, inače pobjeđuje drugi igrač.

Za rješenje preostalih podzadataka, uvodimo oznaku $S(l, r)$ koja označava Grundyev broj stanja u kojem imamo brojeve $a_l, a_{l+1}, \dots, a_r$. Također uvodimo oznake $\text{mex}(\{b_1, b_2, \dots, b_n\})$, predstavljajući najmanji nenegativni broj koji se **ne nalazi** u skupu $\{b_1, b_2, \dots, b_n\}$ i operaciju $\oplus : \mathbb{N}_0^2 \rightarrow \mathbb{N}_0$ koja predstavlja takozvani *binarni xor*. U nekom određenom scenariju, koristeći Sprague-Grundyev teorem, prvi igrač pobjeđuje ako i samo ako $S(l, r) \neq 0$. Preostaje pokazati kako izračunati $S(l, r)$.

Kada igrač na potezu izabere neki broj $x \in \{a_l, \dots, a_r\}$, onda se igra "rastavi" na međusobno nezavisne podnizove $(l, p_{x,1} - 1), (p_{x,1} + 1, p_{x,2} - 1), \dots, (p_{x,k_x} + 1, r)$ gdje su $p_{x,i}$ pozicije x -eva unutar tog podniza. Slijedi da je

$$S(l, r) = \text{mex}(\{S(l, p_{x,1} - 1) \oplus S(p_{x,1} + 1, p_{x,2} - 1) \oplus \dots \oplus S(p_{x,k_x} + 1, r) : x \in \{a_l, \dots, a_r\}\})$$

Promotrimo li izraz, primijetiti ćemo da za neki (l, r) i x iz tog podniza da se navedeni izraz sastoji od nekog prefiksa i sufiksa intervala (l, r) i uzastopnih podnizova koji se nalaze između nekih elemenata koji



su jednaki x , čiji ukupni broj je ograničen s nm , gdje je m broj različitih elemenata u danom nizu.

Kada promotrimo svih q scenarija, uočavamo da je broj uzastopnih podnizova za koje će biti potrebno izračunati odgovarajući Grundyev broj ograničen s $O(m(n+q))$. Pažljivom implementacijom ovog algoritma možemo ostvariti sve bodove na ovom zadatku. Za detalje, pogledajte službenu implementaciju.